The Ledger Knows What the Weights Know: Entity-Level Parametric Memory with Provenance Guarantees for LLM Agents

Caio Vicentino (OpenInterpretability; ORCID 0009-0003-4331-6259)

Working paper, July 2026. License: CC-BY-4.0. Code and benchmark: https://github.com/caiovicentino/cls-ledger

---

Abstract

Long-lived LLM agents need memory, and the field's two answers both lack guarantees: retrieval-based memory cannot promise that superseded facts stay retired or that deleted facts stop influencing answers, while parametric memory (knowledge written into weights) cannot promise bounded forgetting or auditable deletion. We present **CLS-Ledger**, an agent memory architecture in which a symbolic **ledger** of extracted facts — with per-fact provenance and deterministic supersedence — governs a bank of entity-level LoRA **slots** attached to a frozen base model. A router activates at most one slot per query; temporal and multi-hop questions are resolved symbolically in the ledger rather than by the language model. This construction yields three guarantees that we verify empirically: **(1) zero catastrophic forgetting** — with slots inactive the model *is* the base model, and with a slot active a general-capability probe is unchanged (93.8% = base); **(2) physical, surgical unlearning** — deleting an entity's slot changed 0 of 40 unrelated answers, versus 19 of 40 for re-distillation-based deletion; **(3) structural staleness protection** — a fact updated after the last consolidation leaves the consolidated set and is answered episodically. We also release **AgentLife**, a benchmark of synthetic agent "lives" whose answer key is *correct by construction* (a world-state oracle must score 100%, enforced in CI), with typed error taxonomy, instrumented usage, and an adversarial paraphrase protocol. Across seeds, CLS-Ledger beats same-reader BM25 retrieval by +14.6pp (30-day lives, 4/4 paired seeds) and +28.3pp (90-day lives, 2/2), and on a single 365-day seed scores 80.5% vs. 33.9% — non-decreasing with life length while retrieval degrades monotonically. Against a strong embeddings-retrieval baseline under paraphrased questions on an unseen seed, accuracy is comparable (+2.3pp) — we argue, with a route-level decomposition, that the value of parametric agent memory lies in its guarantees and its context-cost profile, not in raw recall accuracy, and we report the negative and null results (LoRA fusion interference, selection-policy ablations) that support this framing.

---

1. Introduction

An agent that works with the same user for a year cannot keep its life in a context window. The dominant engineering answer is retrieval: store episodes or extracted facts externally and inject the relevant ones per query (MemGPT/Letta [1], Mem0 [2], Zep [3]). The emerging research answer is parametric: write experience into weights, via adapters, test-time training, or learned caches. Both families have a missing-guarantees problem, and it is the stated reason parametric memory has not shipped in production systems.

Retrieval-based memory offers no *structural* guarantee that a superseded fact stays retired (stale-

fact rates of 15–40% have been reported for RAG-based memory [4]), and recent auditing work [21] shows that after deletion, "forgotten" information survives almost entirely through the retrieval graph rather than the model parameters — the unlearning boundary is drawn by the database administrator, not the model. Parametric memory has the opposite problem: fine-tuning on experience degrades general capability, deletion requires retraining, and nothing tells the system *what the weights actually know*, so stale parametric answers are undetectable.

Our starting observation is that these are bookkeeping problems, and bookkeeping should not be done by a neural network. **CLS-Ledger** puts a symbolic ledger — atomic facts extracted from the episode stream, with per-fact provenance (source episode, day) and deterministic supersedence — at the center of the architecture, and makes every parametric operation *governed by the ledger*:

- **Consolidation** distills only *current* (non-superseded) facts, selected by a policy over usage and stability, into small per-entity LoRA [25] slots (rank 4), each trained with an anchoring recipe that preserves general capability and calibrated abstention.
- **Routing** activates at most one slot per query — the slot of the queried entity — only when the most relevant *current* fact for the query is one the weights were trained on. A fact updated after the last consolidation has a new identity in the ledger, falls outside the consolidated set, and routes to episodic memory: *the ledger knows what the weights know*, so parametric staleness is impossible by construction.
- **Temporal and compositional resolution** happen in the ledger: "as of day D" questions are answered from ledger snapshots resolved in code, and chain questions ("the deadline of the project X works on") are resolved by parsing the question into an anchor/relation-chain/target lookup (with an LLM parser against a domain schema) and walking the ledger — because, as we show, a small reader model cannot do either reliably even when given all the necessary facts.
- **Unlearning** is file deletion: dropping an entity's slot removes its facts from parametric memory with provably zero effect on other slots and the base model.

Evaluating such a system fairly required a benchmark whose answer key we could trust. Public agent-memory benchmarks have documented reliability problems (a recent audit found 6.4% of one popular benchmark's answer key incorrect, with an LLM judge accepting up to 63% of deliberately wrong answers [16]). We built **AgentLife**: generated agent "lives" (dated episode streams with facts that are stated, updated, revoked, and interleaved with noise) whose answers derive from a programmatic temporal world state. Correctness is enforced, not hoped for: a privileged oracle must score exactly 100% and a deliberately stale oracle must fail every freshness probe — both are CI tests. Scoring is exact string matching over generated accepted/rejected sets (no LLM judge), errors are typed (stale/hallucination/abstain/miss), usage is instrumented for selection experiments, and generation is deterministic per seed across processes.

Contributions. 1. **AgentLife**, a self-validating benchmark for long-lived agent memory with correct-by-construction answers, typed errors, and an adversarial paraphrase protocol (§3). 2. **CLS-Ledger**: to our knowledge the first architecture in which a provenance ledger *governs* entity-level deletable weight slots — deletable adapters exist (SEA [8]) and symbolic ledgers exist (NeuSymMS [12]); the contribution is the coupling — with three construction-level guarantees verified empirically: zero forgetting, zero-collateral unlearning, and structural staleness protection (§4, §5.4). 3. An honest, multi-seed, route-decomposed evaluation showing where

parametric memory genuinely wins (guarantees, context cost, paraphrase-robust behavior of consolidated knowledge) and where it does not (raw recall accuracy versus strong retrieval), including negative results on LoRA fusion and null results on selection policies (§5). 4. A measured case study of **benchmark–system co-adaptation**: our development seed scored 88.6% while unseen seeds average 64.6% — we quantify the gap and describe the paraphrase protocol that exposed which components were template-brittle (§5.6).

2. Related Work

Retrieval-based agent memory. MemGPT/Letta [1], Mem0 [2], Zep/Graphiti [3] and successors store conversation-derived memories externally and retrieve per query; recent product systems add background consolidation ("sleep-time compute" [22]). These systems dominate practice but provide no supersedence or deletion guarantees; temporal knowledge graphs [3] and bi-temporal validity rules (MemStrata [4]) add deterministic staleness control *for retrieval*. We adopt the same deterministic-supersedence philosophy and extend it to govern weight activation.

Parametric memory. Knowledge can be packed into adapters (OPPU's adapter-per-user [5]; document LoRAs; learned KV caches — Cartridges [6], the basis of the Engram system), written by test-time training [23], or consolidated selectively from an agent's stream (EVAF's "memory depth" [7]). Closest to our selection stage, EVAF gates consolidation by surprise and goal-valence into a *single shared* LoRA and explicitly leaves deletion and staleness unresolved. We differ in granularity (per-entity slots), governance (a symbolic ledger decides what the weights know), and guarantees.

Modular unlearning and deletable personalization. SISA-style exact unlearning, adapter-partition methods, and recent systems make deletion a module operation: SEA (Microsoft) [8] keeps per-user "proxy" artifacts (steering vectors + rank-4 LoRA) deletable by construction; TrustErase [9] embeds cryptographic "passports" in LoRA updates for auditable instant unlearning; RRDA [10] routes between edit/suppress adapters per prompt. A position paper signed by much of the continual-learning community [11] argues modular memory is the path to continual agents. Our contribution to this line is granularity and governance: deletion at the *entity* level, routed by a *fact ledger with provenance*, evaluated inside a long-lived agent loop with collateral measured at 0/40 (versus 19/40 for re-distillation).

Neuro-symbolic memory. NeuSymMS [12] builds an agent memory of subject–relation–value triples with rule-based truth maintenance, provenance, and soft deletion — and explicitly contains *no learned weights*. Aeon and MOSS manage symbolic/episodic stores with auditability. We supply the missing half: a ledger of this kind *acting on* parametric memory.

Adapter routing. LoraHub [13], mixtures of LoRA experts, retrieval-augmented adapter selection, and zero-shot parametric-memory routing (PMDRouter [14]) select adapters for *tasks* or *documents*. Our router is comparatively trivial (the ledger names the entity), which is the point: routing by symbolic bookkeeping removes the need to learn what the memory contains.

Benchmarks. LoCoMo [15], LongMemEval [17], MemoryAgentBench [18], BEAM [24] and successors evaluate conversational memory at increasing scale; audits have documented answer-key and judge reliability problems [16], and preference-following studies (PrefEval [19]) show sub-10% adherence within tens of turns. AgentLife differs in being *self-validating* (oracle-100% and stale-oracle-fails as CI tests), in exact typed scoring, and in shipping a paraphrase protocol

that measures benchmark–system co-adaptation directly.

3. The AgentLife Benchmark

AgentLife generates the "life" of a personal-assistant agent over N days (presets: 30/90/365/1000 days). A deterministic generator maintains a temporal world state — entities (people, projects, the user) with typed attributes under four dynamics (stable, updatable, volatile, revocable) — and emits dated episodes verbalizing each event through template banks with deliberate confusables (two "Sofia"s; "Project Auriga" vs. "Project Aurora"). Mid-life **online queries** probe current values (building a per-fact usage log); a **final exam** covers eight query types: current-stable, current-updated (freshness after k updates, superseded values explicitly *rejected*), current-volatile, point-in-time ("as of day D"), 2- and 3-hop composition across episodes >=5 days apart, revocations, and absent facts (never stated; measures hallucination/abstention).

Correct by construction. Answers derive from the world state, never annotation. Three properties are enforced as tests: a world-state oracle scores exactly 100%; a deliberately stale oracle (always answers the first version) scores 0% on freshness probes; every canonical value is verbatim in its source episode. Generation is byte-identical across processes and hash seeds. Scoring is exact matching over per-query accepted/rejected string sets with a typed outcome taxonomy: *correct, stale, hallucination, wrong-value, ambiguous, abstain, miss*. There is no LLM judge.

Adversarial paraphrase protocol. Because templates invite lexical overfitting (in systems and in us), we ship a paraphrase stage: an LLM rewrites each final question with attribute synonyms and restructured syntax under validity constraints (entity names preserved, day references preserved, answers not leaked), and systems are re-evaluated against the original answer sets. §5.6 shows this protocol doing its job — on our own system.

4. CLS-Ledger

Extraction and the ledger. Each episode is distilled by a small LLM into atomic fact cards *(entity, attribute, value, day, source-episode)* against a prescribed attribute schema (the same design choice as Zep's prescribed ontology), with hygiene filters for extraction artifacts. Cards enter the **ledger**: per-fact history with deterministic supersedence (same entity+attribute, later day wins; out-of-order re-statements do not regress), churn statistics, usage counters (charged by mid-life queries), and full provenance.

Consolidation ("sleep"). Periodically (or at end of life), a selection policy chooses current cards to consolidate — the default excludes high-churn ("volatile") facts, and policies over usage/stability/budget are ablatable. Selected cards are grouped by entity; each group trains a rank-4 LoRA **slot** from distilled QA pairs (3 paraphrases per card, revocation-specific QAs, double weight for confusable-twin entities). The recipe that prevents slot collapse: 5 epochs, low LR, and ~10 **anchor rows** per slot (general facts + out-of-scope abstentions). Anchors give each slot calibrated ignorance: a slot answers "unknown" for entities outside its scope, so misrouting fails safe. Slot training is provenance logged: every training row maps to a card, every card to an episode.

Routing and answering. At most one slot is active per query (activation = swapping small LoRA tensors on the frozen base — not formally timed; sub-second in practice). The router

asks the ledger: is the most relevant *current* fact for this question one that was consolidated, and is its entity the question's subject? If yes → that entity's slot answers with no context. If the fact is volatile, or was updated after the last sleep (new card identity not in consolidated set) → episodic route. Temporal questions and relation chains bypass both: a semantic parser (LLM, schema-constrained) extracts *(anchor, relation chain, target, as-of day)* and the ledger resolves the chain and the time slice in code; the reader never performs temporal or compositional reasoning. The episodic route retrieves ledger-resolved fact snapshots (current values, or values as of day D) — not raw episodes — so the reader also never sees conflicting history.

Guarantees by construction. (G1) With no slot active the system *is* the frozen base model; forgetting cannot occur at rest, and slots are trained too small and anchored to move general behavior (verified §5.4). (G2) Deleting a slot cannot change any other answer: other slots and the base are untouched files (verified: 0/40 collateral). (G3) A parametric answer can never be stale: supersedence changes the card identity, which removes it from the consolidated set, which reroutes the query episodically.

5. Experiments

Setup. Reader/consolidation model: Qwen2.5-3B-Instruct [26] (4-bit, MLX) on a single Apple M4; extraction/parsing: gpt-4.1-mini (disk-cached; total API cost for all experiments estimated at ~US$1 from provider dashboards — not instrumented in code). Baselines, all with the *same local reader*: naive continual LoRA on raw episode text; SEAL-lite (episode-level synthetic-QA self-edits, simplified from SEAL [20]); BM25 top-8 RAG; embeddings RAG (text-embedding-3-small, cosine top-8); plus gpt-4.1-mini full-context and RAG as reader-upper-bound references. All response caches are content-addressed; all runs are seeded.

5.1 Parametric baselines: the write-unit matters

On 30-day lives (S-1), answering *with no context at all*: naive LoRA 15.9% (raw text does not become queryable knowledge; 44/59 abstentions), SEAL-lite 20.5% with 10 stale errors (episode-level self-edits memorize superseded mentions), CLS consolidation 31.8% with 3 stale errors and 75%/75% on stable/updated facts — 3× SEAL-lite on updated facts. State-level consolidation with supersedence, not episode-level editing, is the right write unit.

5.2 Main results: multi-seed, paired

Slots-routed CLS-Ledger versus same-reader BM25 RAG (final-exam accuracy, mean ± sd over seeds; paired by seed):

	CLS-Ledger	BM25 RAG	paired
S (30 days, 4 unseen seeds)	64.6% ± 6.7	50.1% ± 4.1	4/4 wins, +14.6pp
M (90 days, 2 unseen seeds)	71.1% ± 1.7	42.8% ± 2.6	2/2 wins, +28.3pp
L (365 days, 1 seed)	80.5%	33.9%	+46.6pp

Development seeds (S-1, M-1) are excluded from the means at both scales for symmetry — they

are the seeds error analysis was performed on (S-1: 88.6%; M-1: 74.4% vs. RAG 37.8%, also a paired win). CLS-Ledger wins every unseen paired seed: 6/6 pooled across scales (two-sided sign test p = 0.031; per-scale counts are individually too small for significance — a caveat we state rather than hide, and the reason the multi-seed protocol ships with the benchmark).

The scale trend is the headline: CLS accuracy does not decrease from 90 to 365 days (72.2 -> 80.5; the 365-day point is a single seed, stated as such) while retrieval degrades monotonically; facts with 2+ updates score 92.3% at 365 days — consistent with supersedence mattering more in longer lives, though this is an interpretation of few points, not a fitted trend. Online (mid-life) accuracy at L: 84.4%.

5.3 Against strong retrieval: parity, not dominance

On an unseen seed (S-2) with the strong embeddings baseline and the same reader: CLS 72.1% vs. EmbRAG 62.8% on template questions (+9.3pp); CLS 58.1% vs. 55.8% under paraphrase (+2.3pp — statistical parity at n=43). We therefore do *not* claim a general accuracy advantage over well-built retrieval. The claims that survive: the guarantees (§5.4), the context-cost profile (~39–45% of queries answered with zero context tokens, recomputed per run from route logs), and the robustness signature (§5.6).

5.4 Guarantees, measured

Success criteria for G1 and G2 were pre-registered before the slot architecture was built (<=1% probe degradation; <1% unlearning collateral); everything in this subsection is a test of those pre-registered criteria.

Forgetting. General-capability probe (16 items, disjoint from all training): base 93.8%; naive-LoRA/SEAL adapters 87.5% (with memory-bleeding errors — a benchmark company name answered for "currency of the UK"); monolithic CLS adapters: 68.8% without anti-forgetting replay and 75.0–81.2% with it across development versions (the re-runnable HEAD artifact scores 75.0%, reports/forgetting_probes.json; earlier points are log-derived); **slots-routed: 93.8% with slots at rest and 93.8% with a slot active — 0.0% degradation**, meeting the pre-registered <=1% criterion.

Unlearning. Deleting one entity (7 cards): target queries flip to "unknown"; **0 of 40 other answers changed** (pre-registered <1% collateral), versus 19/40 changed under re-distillation-based removal — retraining without isolation is not surgical, slot deletion is.

Fusion negative result. Exact multi-slot fusion (rank concatenation, sum of deltas) does not merely degrade — it collapses the model: the 13-slot fused adapter scores **0.0%** on the general-capability probe and **2.3%** answering the final exam weights-only (reports/forgetting_probes.json, reports/S-1-slots-fused-weightsonly.json); the historical full-system fused run salvaged 52.3% (log-derived) only through its episodic/symbolic routes. Summed deltas amplify the slots' shared format component. Isolated routed activation, not fusion, is what makes entity-slots viable; we report this so others skip the failure.

5.5 Selection policies: a null result under loose budgets

Under a 40-card capacity budget on 90-day lives (2 unseen seeds), usage-gated selection ("hot"), stability-gated ("stable") and random selection are statistically inseparable (77.1 / 74.7 / 77.1

mean). Usage-gating *is* the efficiency champion — 35 consolidated cards matched the full 139-card consolidation — but with budgets that cover most queried facts, *which* facts you consolidate does not move end-to-end accuracy. Discriminative selection experiments need binding budgets and cost-charged protocols; we flag this as an evaluation-design lesson.

5.6 Co-adaptation, paraphrase, and what is actually robust

Our development seed scored 88.6% (S-1); unseen seeds average 64.6% — a 24pp co-adaptation gap, quantified because every v2/v3 fix was derived from error analysis on that seed. The paraphrase protocol then decomposed the system: with the v3 lexical question parser (measured at commit da1b072, before the parser was replaced; log-derived, not re-runnable at HEAD), paraphrased questions dropped CLS to 56.8% — multi-hop fell to 0% (the lexical chain resolver died) and point-in-time to 20% (the "As of day" regex died) — **while the parametric route was unaffected** (updated/volatile/revoked at 100% under paraphrase). Consolidated knowledge is semantically robust; symbolic machinery was template-brittle. Replacing the lexical parser with a schema-constrained LLM parse (resolution still in the ledger) recovered paraphrased accuracy to 75.0% and *improved* template accuracy to 93.2% on S-1 (72.1% on the unseen seed). Epistemic status, stated plainly: the paraphrase protocol was designed *after* the lexical failures were observed, and the semantic parser is a post-hoc fix validated on two seeds; the pre-registered claims of this paper are the guarantees of S5.4, not the parser recovery, which awaits pre-registered replication on fresh seeds. Retrieval baselines, for calibration, are paraphrase-invariant (BM25: 52.3% both ways). One validity threat we disclose rather than hide: the paraphrases are generated by the same model family (gpt-4.1-mini) that serves as the semantic parser, so part of the parser's recovery could reflect distributional familiarity; re-running the protocol with a disjoint paraphrase model is future work.

5.7 Reader-model ceiling

gpt-4.1-mini as reader adds ~20pp to retrieval baselines on identical data (58.5% vs. 37.8% RAG at 90 days). Two implications: absolute numbers in this paper are compressed by the 3B reader, and — the reason our symbolic resolution exists — a 3B reader demonstrably cannot compose two retrieved facts or do timeline arithmetic even when given everything it needs; the architecture moves exactly those steps into code.

6. Limitations

AgentLife is synthetic and template-verbalized; extraction is far easier than on real dialogue, and facts are atomic. The attribute schema is prescribed (as in deployed systems, but it bounds generality); the semantic parser inherits the schema. All experiments use one 3B reader and <=26 entities per life; per-entity slots do not capture cross-entity or procedural knowledge, and the base model never learns — the system becomes better *informed*, not more capable. A preprocessing asymmetry should be read as a confound: CLS's ingestion uses gpt-4.1-mini for fact extraction and question parsing (a far stronger model than the 3B reader), while the BM25 baseline uses no external model at all — a "BM25-over-extracted-cards" baseline that isolates this effect is future work (the embeddings baseline partially controls for it, since it also uses an external API model). Multi-seed replication covers the main comparison (6 paired seeds) but not yet every ablation; single-seed numbers are marked as such. Co-adaptation was measured

and partially corrected, not eliminated: results on external benchmarks (e.g., LongMemEval) are future work.

7. Ongoing Work

The ledger's complete supersedence history makes two extensions natural: **quantitative induction** (frequency/tendency/habit questions answered by calibrated symbolic aggregation over fact histories — the aggregation prerequisite that retrieval-based systems systematically fail) and **dispositional memory** (consolidating user preferences as behavior that persists across sessions without context, with slot-level deletability). Both come with AgentLife extensions whose ground truth is generated with the world state.

8. Conclusion

Parametric memory for agents does not need to beat retrieval at recall to matter; it needs guarantees retrieval cannot give, at comparable accuracy and lower context cost. Making a symbolic provenance ledger govern entity-level weight slots delivers three such guarantees by construction — zero forgetting, surgical deletion, structural freshness — and our measurements show the price in accuracy against strong retrieval is small to nil while the benefits are absolute. The bookkeeping of memory belongs in code; the knowledge can live in weights; and the system should always know which is which.

References

[1] C. Packer et al. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023. [2] Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. arXiv:2504.19413, 2025. [3] Zep: A Temporal Knowledge Graph Architecture for Agent Memory. arXiv:2501.13956, 2025. [4] MemStrata: Temporal Validity in Retrieval Memory — Eliminating Stale-Fact Errors. arXiv:2606.26511, 2026. [5] Z. Tan et al. Democratizing Large Language Models via Personalized Parameter-Efficient Fine-tuning (OPPU). EMNLP 2024; arXiv:2402.04401. [6] S. Eyuboglu et al. Cartridges: Lightweight and General-Purpose Long-Context Representations via Self-Study. arXiv:2506.06266, 2025. [7] H. Han. Memory Depth, Not Memory Access: Selective Parametric Consolidation for Long-Running Language Agents. arXiv:2606.26806, 2026. [8] C. Schneider, P. Schoenegger, B. Bariach. Separable Expert Architecture: Toward Privacy-Preserving LLM Personalization via Composable Adapters and Deletable User Proxies. arXiv:2604.21571, 2026. [9] R. Hendrix et al. TrustErase: Auditable Instant Machine Unlearning with Passport-Embedded Representations. arXiv:2606.17122, 2026. [10] B. Zhang, Y. Huang. When to Write and When to Suppress: Route-Specialized Dual Adapters. arXiv:2606.14668, 2026. [11] Position: Modular Memory is the Key to Continual Learning Agents. arXiv:2603.01761, 2026. [12] M. Sultan, N. Thuraisamy, V. Rajaratnam. NeuSymMS: a neuro-symbolic agent memory with rule-based truth maintenance and provenance. arXiv:2605.17596, 2026. [13] C. Huang et al. LoraHub: Efficient Cross-Task Generalization via Dynamic LoRA Composition. COLM 2024; arXiv:2307.13269. [14] F. Ji et al. Parametric Memory Decoding for Zero-Shot Routing in LoRA-Based External Parametric Memory. arXiv:2607.04118, 2026. [15] A. Maharana et al. Evaluating Very Long-Term Conversational Memory of LLM Agents (LoCoMo). arXiv:2402.17753, 2024. [16] Penfield Labs. We Audited LoCoMo: 6.4% of the Answer Key Is Wrong and the Judge Accepts up to 63% of Intentionally Wrong Answers. 2026. github.com/dial481/locomo-audit.

[17] D. Wu et al. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory. ICLR 2025; arXiv:2410.10813. [18] Y. Hu et al. MemoryAgentBench: Evaluating Memory Agents. ICLR 2026; arXiv:2507.05257. [19] S. Zhao et al. Do LLMs Recognize Your Preferences? Evaluating Personalized Preference Following in LLMs (PrefEval). ICLR 2025; arXiv:2502.09597. [20] A. Zweiger et al. Self-Adapting Language Models (SEAL). NeurIPS 2025; arXiv:2506.10943. [21] A. Raeesi, H. Roed. Auditing Forgetting in Limited Memory Language Models. arXiv:2607.00605, 2026. [22] K. Lin et al. Sleep-Time Compute: Beyond Inference Scaling at Test-Time. arXiv:2504.13171, 2025. [23] Y. Sun et al. Learning to (Learn at Test Time): RNNs with Expressive Hidden States. arXiv:2407.04620, 2024. [24] M. Tavakoli et al. BEAM: Beyond a Million Tokens. ICLR 2026. github.com/mohammadtavakoli78/BEAM. [25] E. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. ICLR 2022; arXiv:2106.09685. [26] Qwen Team. Qwen2.5 Technical Report. arXiv:2412.15115, 2024.

Acknowledgments: experiments, code, and drafting were carried out with the assistance of Claude (Anthropic). All claims were verified against run artifacts in the linked repository.

Reproducibility: every number in this paper corresponds to a JSON artifact under `reports/` *in the repository (generated by seeded, cached, deterministic runs), except where explicitly pinned to a development-log commit in the text; the benchmark's self-validation suite (oracle-100%, stale-oracle-fails, determinism across hash seeds) runs as tests.*